iHuman Lab Research Manual

Hemanth Manjunatha

2026-10-15

Table of contents

Welcome 7
- Acknowledgment . . . 7

Introduction 8
- Who `is` this lab manual for? . . . 8
- Why is it worth my time to read through the manual? . . . 9
- What `isn't` this lab manual? . . . 9

I Writing principles 10
- Using GPT . . . 12

Writing articles 14
- Why choose LaTeX? . . . 14
- Resources . . . 14

Making diagrams 16
- Links . . . 17

Making presentations 18
- General guidlines . . . 18
- Preparing the slides . . . 19
- Links . . . 20

Making posters 21
- How to prepare a `classic` scientific poster . . . 21
  - Guidelines . . . 21

II Coding practices 23

Development setup 26
- Visual Studio Code (VSCode) . . . 26
- Conda environment management . . . 26
- Ruff for code linting . . . 27
- Pytest for testing . . . 27

Pre-commit hooks . . . 27
GitHub for version control . . . 27

Code cleanliness **29**
Meaningful naming . . . 29
Use descriptive names . . . 29
Use consistent naming conventions . . . 29
Small and focused functions . . . 30
Single responsibility principle . . . 30
Avoid deep nesting . . . 30
Clear and consistent formatting . . . 31
Follow PEP 8 . . . 31
Use docstrings . . . 32
Error handling . . . 32
Use exceptions appropriately . . . 32
Avoid empty `except` clauses . . . 33
Refactoring . . . 33

Decoupled code **34**
Understanding decoupling in code . . . 34
Principles of decoupled Python code . . . 34
Single responsibility principle (SRP) . . . 34
Dependency injection . . . 35
Use interfaces and abstract classes . . . 36
Separation of concerns . . . 36
Conclusion . . . 37

Code testing **38**
Basic test structure . . . 38
Writing tests with `pytest` . . . 38
Basic assertions . . . 38
Using fixtures . . . 39
Parameterized tests . . . 39
Testing for exceptions . . . 40
Customizing test output . . . 40
Advanced features of `pytest` . . . 40
Plugins . . . 40
Fixtures with scope and autouse . . . 41
Integrating `pytest` into the research workflow . . . 41
Continuous integration (CI) . . . 41
Code coverage . . . 42
Test documentation . . . 42
Conclusion . . . 42

Code documentation 43
Importance of documentation . . . 43
Types of documentation . . . 43
Code comments . . . 43
Docstrings . . . 44
README files . . . 45

Usage 46
Tools for generating documentation . . . 46
Sphinx . . . 46
Best practices for documentation . . . 47
Write clear and concise descriptions . . . 47
Update documentation regularly . . . 47
Use consistent style . . . 47
Include Examples . . . 47
Document edge cases and limitations . . . 47
Conclusion . . . 47

Project structure 48

Using Git 50
Set up Git . . . 50

III Tutorials 53

Python code structure 54
Overview . . . 54
Project structure . . . 54
1. Modules . . . 54
Creating a module . . . 55
2. Packages . . . 55
Creating a package . . . 55
3. Subpackages . . . 56
Creating a subpackage . . . 56
4. Importing and using modules . . . 57
Example main script . . . 57
Importing within the same directory . . . 58
Relative imports . . . 58
5. Understanding `__init__.py` . . . 59
Importance of `__init__.py` . . . 59
Example of `__init__.py` . . . 59
Using the package . . . 60

6. Running the code . . . 60
Expected output . . . 60

Python resources **62**
General resources . . . 62
Getting started . . . 62
Conda . . . 62
Scientific computing . . . 63
Open source . . . 63
PyVideo . . . 63
WTF Python . . . 64
Python Tutor . . . 64
Visualizing NumPy . . . 64
Websites . . . 65
PyCon . . . 65
Pytorch . . . 65
PyTorch Lightning . . . 65
Other PyTorch related links . . . 66
Ray . . . 66
Key features of Ray . . . 66
Resources and documentation . . . 67

BCI resources **68**
Videos . . . 68
GitHub links . . . 69
Books . . . 69
MNE Python . . . 69
Lab Streaming Layer (LSL) . . . 70

EEG resources **72**
EEG glossary . . . 72
Setting up EEG recordings . . . 72
EEG analysis steps . . . 72
MNE (MNE-Python) . . . 73
EEG databases . . . 73
Key papers . . . 74
GitHub links . . . 74
Analysis related repositories . . . 74
Useful libraries and links . . . 74
Deep learning with physiological signals . . . 75

ROS resources **76**

Lab resources **77**

Checklist and signature page **78**

Welcome

We are glad that you have decided to join **I**ntelligent **Hu**man-**Ma**chine **N**exus Lab (iHuman Lab)! We are really excited to have you as a part of our team and we will do what we can to make sure that you have a great time in our lab. We hope you will learn a lot about human and machines, and neuroscience, develop new and useful skills (coding, data analysis, writing papers, giving talks), make new friends, and have a great deal of fun throughout the whole process!!

This lab manual is crafted as your essential companion, designed to acquaint you with our lab's standard workflow. Within these pages, you will find detailed explanations writing and coding practices, learning resources for Python, BCI, and physiological data processing. These workflows ensure smooth integration into our research.

Acknowledgment

This lab manual draws inspiration from various sources, and in some sections, content has been adapted or reproduced from these sources. We acknowledge the following works for their influence:

Context Dynamics Lab, The Memory Modulation Lab, Aly Lab, Peelle Lab, Ritchey Lab, Kemp Lab

This lab manual is licensed under the Creative Commons Attribution 4.0 International License. You are free to share, adapt, and redistribute the material in any medium or format, provided appropriate credit is given to the original authors and the source is properly cited.

Introduction

This lab manual is intended to provide a crash course in doing research in the iHuman Lab. It describes standard workflows we carry out in the lab. However, these are not rigid rules, they are meant to be flexible guidelines.

Who `is` this lab manual for?

Every new lab member should read the latest version of this lab manual in detail and reference it later as needed. Periodically throughout the document, you will see callouts with listed `Task` items.

Completing your read-through entails:

- Reading the contents of the manual
- Asking current lab members about any confusing aspects
- Completing the relevant `Task` items. You will also see non-task NOTE items; these provide helpful tips and additional commentary on the nearby text.

This lab manual is meant to be a `living document.` All lab members are welcome (and encouraged!) to submit edits that improve the content, clarity, and overall helpfulness of this document at any point throughout their tenure in the lab. Knowledge is ever changing, you might come across new tools, methods, and resources which might help you and future lab members. You can share that knowledge through this manual

> **Task**
>
> Upon reading through this lab manual for the first time, please update the document to include your name in the Lab members section. Importantly, be sure to fork the `GitHub` repository, make your edit on your personal fork, and submit a pull request with your update.

Why is it worth my time to read through the manual?

Aside from pursuing your own curiosity, automating aspects of your research can significantly reduce the mundane tasks that consume valuable time and effort. By implementing automated processes, you free up mental energy and resources that can then be redirected towards exploring more complex and thought-provoking questions within your field.

This manual serves as a foundational guide to help you establish an efficient workflow for automating your research tasks. By following these steps, you will streamline data collection, analysis, and synthesis processes, allowing you to focus more on formulating innovative hypotheses, designing insightful experiments, and interpreting results in novel ways.

What `isn't` this lab manual?

This lab manual should not be regarded as a rigid set of rules to be strictly adhered to. Rather, it serves as a collection of established workflows that have been utilized by others and found beneficial. It acknowledges that individual preferences and circumstances vary, and therefore, not every aspect of these workflows may align perfectly with your own methods or preferences.

The workflows outlined here are meant to be flexible tools that can be adjusted and refined according to the unique requirements of your research projects.

Part I

Writing principles

Research is not merely about numbers and experiments; it's about the transformative power of sharing discoveries with the world. Beyond coding, mastering the art of writing is essential for every researcher. Yet, for many, especially non-native English speakers like myself, writing can seem like scaling a daunting mountain. Here's a principles drawn from my own journey—from uncertainty to embracing writing as a vital research skill:

Recognize the importance of writing: Clear and effective communication is key to making your research count. Writing isn't just about putting words on paper—it's about bridging the gap between complex data and its real-world impact, making sure your work resonates and influences others.

Acknowledge challenges: Writing can feel intimidating, especially if English isn't your first language. It's okay to feel daunted. Embrace these challenges as chances to grow and enrich your academic journey.

Embrace the purpose: Despite the initial hurdles, remember that effectively sharing your findings is crucial. When you articulate your ideas clearly, you not only boost your credibility but also amplify the importance of your research.

Overcome self-doubt: It's common to doubt your writing skills, but don't let these fears hold you back. Start small, set achievable goals, and build confidence through practice. You've got this!

Develop a writing routine: Find a writing schedule that works for you—whether it's daily or weekly. Consistency is key to gaining momentum and improving your writing skills over time.

Find your voice: As you refine your writing, focus on developing a voice that's uniquely yours. Being authentic in your writing not only engages readers but also showcases your passion for research beyond technical details.

Celebrate progress: Take pride in every step of your writing journey—whether it's completing a draft, receiving positive feedback, or submitting a manuscript. Celebrating these wins boosts your motivation and keeps you moving forward.

Persist and persevere: Writing is a skill that improves with persistence. Embrace challenges as opportunities to learn and refine your craft. Each round of edits and revisions will sharpen your writing, making it more compelling and impactful.

```
Embrace writing not just as a tool, but as a catalyst for transforming your
research into a narrative that inspires, informs, and shapes the future of
your field. Through dedication and resilience, your journey as a researcher
and writer will illuminate new pathways of discovery and impact.
```

Using GPT

Using GPT as a writing tool can be immensely beneficial, but it's important to approach it with care and use it as a supportive resource rather than a crutch. Here's how to leverage ChatGPT effectively while maintaining control over your writing process:

1. **Generate ideas and inspiration** ChatGPT can help spark ideas and provide inspiration for your writing projects. Use it to brainstorm topics, explore different angles, or overcome writer's block by generating prompts or outlines.
2. **Improve clarity and structure** Ask ChatGPT for suggestions on structuring your writing. It can provide guidance on organizing your thoughts logically, creating smooth transitions between paragraphs, or refining the overall flow of your piece.
3. **Enhance language and style** Use ChatGPT to improve the clarity and coherence of your sentences. It can offer alternative word choices, help with phrasing, and suggest improvements in grammar and syntax.
4. **Editing and proofreading** Use ChatGPT to review your draft for spelling errors, grammatical mistakes, and punctuation issues. It can serve as a preliminary editor, but final proofreading should always be done by you.
5. **Feedback and revision suggestions** Seek feedback from ChatGPT on the overall effectiveness of your writing. It can provide insights into areas that may need further development or clarification.

> ! Important
>
> - `Maintaining ownership of your ideas and writing style` while using GPT is crucial to preserving the authenticity and integrity of your work. As a tool, ChatGPT offers valuable assistance in refining and enhancing your writing, but it should not overshadow your unique voice as a writer. Your critical thinking skills play a pivotal role here; they allow you to evaluate the suggestions provided by ChatGPT thoughtfully and decide how best to integrate them into your writing.
> - While it can generate text based on input and provide suggestions, not all responses may be equally relevant or of high quality. It's essential to scrutinize the responses, considering factors such as accuracy, coherence, and appropriateness for your intended audience. `Evaluate each suggestion critically to ensure it aligns with the tone, style, and purpose of your writing.`
> - By leveraging GPT mindfully as a writing tool, you can indeed enhance your writing skills and streamline your workflow. It can help you achieve clearer and more polished pieces by offering insights into structure, language refinement, and even creative ideation. However, it's paramount to maintain a balanced ap-

proach. **Avoid relying solely on GPT for the creative and intellectual aspects of your writing process. Remember, your originality and ability to engage with your subject matter in a meaningful way are what ultimately define your work as uniquely yours.**

Writing articles

If you're diving into research or getting started with writing scientific articles, LaTeX is your friend. LaTeX (pronounced "LAY-tech") is not just any tool—it's a powerful typesetting system that helps you create beautifully formatted documents, perfect for academic and scientific writing. In our lab, we almost exclusively use LaTeX for most of the documents.

Why choose LaTeX?

- **Polished Documents**: It makes your research papers and lab manuals look super professional with neat equations, figures, and references.
- **Focus on Content**: You get to focus on what really matters—your research—without getting bogged down by formatting headaches.
- **Collaboration Made Easy**: LaTeX plays well with others. It's perfect for working on documents with classmates or colleagues, thanks to its compatibility with tools like Overleaf.

> **Task**
>
> 1. Head over to Overleaf and sign up—it's free!
> 2. Click on "New Project" and choose "Blank Project". This sets you up with a clean slate to start writing your document.
> 3. Overleaf shows your document in two parts: on the left, you write your LaTeX code (don't worry, it's easier than it sounds!), and on the right, you instantly see how it looks as a document.

To get started with LaTeX in Overleaf, follow the tutorials below

For more detailed tutorials, please see the Overleaf Documentation

Resources

These are some of the books (&video) which have personally helped to write better:

1. Style: Toward Clarity and Grace, Joseph M

2. They Say I Say by Graff and Birkenstein
3. University of Manchester, Academic Phrasebank

University Chicago Writing Workshop

> **Task**
>
> 1. Go through the book `Style: Toward Clarity and Grace, Joseph M`.
> 2. Watch the video `LEADERSHIP LAB: The Craft of Writing Effectively` in the above set of videos.

Making diagrams

Welcome to the world of scientific diagramming, where creativity meets clarity and innovation thrives through visualization. As researchers, we understand the pivotal role that diagrams play in transforming complex data into compelling narratives that resonate with audiences. Whether you're sketching out a hypothesis, illustrating experimental results, or mapping intricate processes, the tools at your disposal can make all the difference in how effectively your research is communicated.

In this section, we'll explore a variety of open-source tools designed to empower you in your quest for impactful visual storytelling. Each tool offers unique capabilities—from precise vector graphics to dynamic data plotting—and is tailored to meet the diverse needs of scientific communication. It's important to note that while we will cover several powerful tools, this list is not exhaustive. The field of open-source software continues to evolve, offering new and innovative solutions for researchers every day.

1. **Draw.io**
    - Draw.io is a versatile web-based diagramming tool that offers a user-friendly interface and a comprehensive library of templates and shapes. It is designed to facilitate the creation of various types of diagrams, from flowcharts and organizational charts to network diagrams and technical drawings.
    - Ideal for researchers who need to quickly sketch out ideas, brainstorm concepts, or map out processes. Its collaborative features make it suitable for teams working together on visual representations of complex data and workflows.
2. **Inkscape**
    - Inkscape is a powerful open-source vector graphics editor that allows for precise and scalable artwork creation. It supports advanced features such as object manipulation, fill and stroke properties, and extensive text editing capabilities.
    - Researchers can utilize Inkscape to produce publication-quality figures, technical illustrations, and graphical abstracts. Its ability to handle intricate details and export to various formats makes it indispensable for creating visually appealing diagrams that meet academic standards.
3. **TikZ**
    - TikZ is LaTeX packages renowned for their versatility in generating high-quality graphics directly within LaTeX documents. TikZ provides a flexible and programmable framework for creating detailed graphics.

- Researchers proficient in LaTeX can leverage TikZ to integrate complex mathematical plots, chemical structures, and data visualizations seamlessly into their academic papers and presentations. This tools excel in producing precise and publication-ready diagrams with LaTeX's typesetting capabilities.

4. **PowerPoint**

    - Microsoft PowerPoint is a widely-used presentation software that also offers basic diagramming capabilities. While not open-source, it is included here for its accessibility and familiarity among researchers.
    - Researchers who prefer a straightforward approach to creating diagrams within a presentation environment. PowerPoint is suitable for creating simple flowcharts, timelines, and conceptual diagrams directly integrated into slide presentations.

> **Note**
>
> Eventhough we use Drawio as our main tool, select the tool that best suits your specific needs based on the type of diagram you need to create and your comfort level with the software. Experiment with different tools to find one that enhances your ability to communicate your research effectively and professionally.

Links

The true essence of tools emerges only when they are wielded by an artist—the researcher. A diagramming tool, on its own, is a set of digital functionalities and features. It is through the skilled hands of the scientist that these tools transcend mere utility to become vehicles of clarity and innovation.

Here are some links/tutorials which help you hone your diagram making skills. Please note that these resources are not exhaustive.

- General design tips
- Inkscape tutorials

Making presentations

Creating an effective presentation involves careful planning and thoughtful execution to ensure your audience remains engaged and gains a clear understanding of your message.

Over the years, I have come to appreciate these principles while making presentations

> **Note**
>
> - A scientific presentation is about making your audience smarter, not making yourself seem smart.
> - A blind high schooler and a deaf high schooler should both get the same message out of your presentation.

Here's a brief guide based on best practices for delivering impactful presentations:

General guidlines

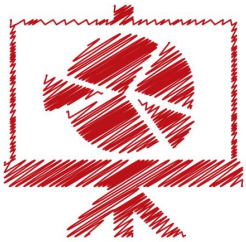
- Face the audience, speak to them!
- Don't read the slides
- Don't put to much writing
- Don't use color maps that are not blind-color friendly
- Don't write long equations, no one can follow those
- Please respect your time, it's the audience time and other speakers time too.
- Keep the number of slides under the number of total minutes
- Keep the number of messages per slide to one, or at most…one
- Don't be surprised by the next slide (like, "crap now I have to explain this slide")
- Don't put on a full manuscript table and then say "I know you won't be able to read this"
- Also, don't skip slides in front of your audience like no one is noticing. "Sorry, just want skip these here for the sake of time… Ok… Oh, one more. Yes. As I was saying…"
- If you show an equation, give an intuition for what it's telling you. The title of a data slide should (almost always) be a SINGLE SENTENCE that tells the audience what you concluded from the data

Preparing the slides

- If you don't explain it, don't put it on the slide.
- Give slides titles that state the conclusion of the data
- Add animations!
- Convince the audience what you plan to tell them is worth the effort of listening VERY early in the talk. And one "idea" per slide (this doesn't mean Fig1A-H copy pasted from your manuscript).
- Make sure figures (and their axes labels) are big enough (don't make me squint)
- Physically point to the diagram during explanations
- More visuals than text!
- Statements to avoid:
  - "I know there's a lot of data here, but I'm only going to talk about some of it" - don't put data there if you don't talk about it
  - "Um" - try to speak clearly
  - "You may not be able to see this, but…" - make the figure bigger
- And if it's for you to show things, don't put any words. People CANNOT READ AND LISTEN at the same time. So many people can't understand this simple fact.
- Have a conclusion slide that summarizes the key takeaways from the presentation. During Q&A, stay on this conclusion slide (allowing it to sink in to the audience) instead of a slide that simply says "Thank you", or "Questions?" or something like that.
- Don't overestimate the background knowledge of your audience. Spend a few extra minutes explaining the background of your research, why it's interesting, and defining any special terms you use. As long as you're not patronizing, everyone will appreciate it.
- Don't say "bear with me" or "this looks complicated but…" or anything that implies that your audience is not good, smart, quick enough to get you…
- Have backup slides. You can anticipate or will experience common questions or follow-on detail/data that audiences will ask.
- Describe both axes of a graph on every single slide. You know the data. Your audience doesn't. Orient them.
- I TRY to maintain one idea per slide. New idea or piece of data? New slide.
- Bullet points are for suckers. Use an image, and talk over it. This is my "spray and pray marketing" slide.

Links

> **Task**
>
> Complete the Linkedin course on making presentations: PowerPoint: Designing Better Slides



Making posters

In poster sessions, students and researchers try to convey their research to an audience walking through a busy hall. To reach and engage that audience, posters should be eye-catching, to the point, and easy to understand. This section is from here

How to prepare a `classic` scientific poster

While the status quo of scientific posters is far from perfect (see video above), academic supervisors, organizations and conference may require scientific posters in a pre-defined format. Below you will find a few tips to prepare a scientific poster that meets those expectations.

Guidelines

Basics

- Title and other required sections are present
- Complete author affiliation & contact information is included
- The poster conforms to the requirements of the conference or program where it will be presented
- Font is consistent throughout
- Spelling is correct throughout
- Grammar is correct throughout
- Acronyms are defined on first use
- Content is appropriate & relevant for audience

Design

- All text can be easily read from 4 feet away
- Flow of the poster is easy to follow
- White space used well
- Section titles are used consistently
- Images/graphics are used in place of text whenever possible
- Bullet points/lists are used in place of text whenever possible

- All images are relevant and necessary to the poster
- Charts are correct – i.e. appropriate type for data, data is correct & correctly represented
- Text color and background color are significant in contrast for easy reading
- Background color doesn't obscure or dim text
- Images are clear, not pixilated or blurry

Content

- The "story" of the poster is clear
- The content is focused on 2-3 key points
- Title is clear & informative of the project
- Problem, or clinical question, is identified and explained
- Current evidence related to project is listed
- Objectives are stated
- Methods are described
- Results are presented
- Conclusions are stated
- Implications to practice and to other professions are presented
- References are listed
- All content is relevant and on the key points
- Content is not duplicated in text and graphics

Oral presentation

- Presenter greets people
- Presenter is able to give a concise synopsis of poster
- Presenter is able to explain all diagrams and sections
- Presenter speaks fluently – i.e. doesn't stumble, leave sentences/thoughts hanging
- Presenter has questions to ask viewers

More tips:

Part II

Coding practices

Effective code management is a game-changer in research, especially for data analysis and simulations. Follow this general guide to help you organize your projects and boost your productivity:

1. **Establishing a structured development environment**

    - Start by setting up Miniconda or Anaconda to manage your development environments effectively. By doing this, you can create isolated environments for each of your projects, ensuring that dependencies are kept separate and conflicts are avoided.
    - Customize and tailor separate environments for each of your projects using conda. This approach ensures that each project has its own specific set of dependencies, which helps in maintaining a clean and well-organized workflow.
    - Use tools like ruff to enforce consistent code formatting across your projects. Consistent formatting not only makes your code look more professional but also enhances its readability and maintainability, making your coding practices more efficient.

2. **Organizing and structuring your codebase**

    - Design a clear and logical folder structure for your project, such as including `data/`, `notebooks/`, `src/`, `tests/`, and `scripts/` directories. This organization will make it easy for you to navigate your project, locate files quickly, and keep everything neatly arranged.
    - Leverage git for version control and use github for managing collaborative development. This setup allows you to track changes, collaborate seamlessly with others, and revert to previous versions if needed, ensuring that your project evolves smoothly.
    - Include thorough comments in your code and create a detailed `README.md` file. This documentation will help explain your project's objectives and usage, making it easier for others to understand and contribute to your work. Clear documentation acts as a friendly guide for anyone interacting with your project.

3. **Effective data management practices**

    - Store your raw data in a `data/raw/` folder and place your processed data in a `data/processed/` folder. This method helps keep your data organized and clearly distinguishes between original and modified datasets, which maintains data integrity throughout your project.
    - Use robust data management tools to track different versions of your data. By managing versions effectively, you ensure that your results are reproducible and that you can trace back any changes, supporting scientific rigor and maintaining a clear record of your data's evolution.

4. **Ensuring reproducibility and continuity**

    - Share your environment configurations with others to facilitate a smooth setup of your project on their systems. This practice helps ensure that your project is reproducible and that others can replicate your work without any difficulties.

- Regularly update your documentation and Jupyter Notebooks to reflect the latest changes and insights. Keeping these resources current makes it easier for others to follow your work and for you to share your findings, aiding in knowledge transfer and enhancing the accessibility of your project.

5. **Facilitating collaborative excellence**

   - Foster a culture of quality assurance by encouraging regular code reviews within your team. This practice helps catch potential issues early, incorporate feedback, and continuously improve the quality of your code, leading to better results and stronger collaboration.
   - Set up effective communication channels to discuss ideas, share insights, and solve problems together. Strong communication is essential for successful collaboration and helps build a supportive and innovative team environment.

6. **Sustaining long-term project integrity**

   - Stay updated with the latest tool and library advancements to keep your project performing at its best. Embracing new updates and improvements ensures that your project remains current, reliable, and optimized for performance.
   - Utilize issue tracking tools to address bugs and enhancements promptly. Proactively managing issues helps maintain the stability and growth of your project, ensuring it remains robust and continues to meet its goals.

Development setup

In the world of professional Python development, setting up a robust and efficient development environment is crucial for productivity and code quality. This tutorial provides a comprehensive guide to setting up a Python project using industry-standard tools and practices. We'll walk you through the process of installing Conda for environment management, configuring Visual Studio Code (VSCode) for a seamless development experience, and organizing your project structure effectively. Additionally, we'll cover how to manage dependencies with Conda and pip, ensure code quality with Ruff for linting and formatting, and automate workflows using pre-commit hooks. By following these steps, you'll establish a solid foundation for developing, testing, and maintaining your Python projects. For each step, we've linked to practical YouTube videos to enhance your understanding and provide visual guidance.

All these topics are covered here:

For more detailed tutorial see each sub-section.

Visual Studio Code (VSCode)

Visual Studio Code (VSCode) is a versatile, open-source code editor developed by Microsoft. It provides essential features for Python development, including debugging, linting, and integrated terminal support.

Conda environment management

Conda is a powerful package and environment management system that helps manage dependencies and isolate project environments. It supports both Python and non-Python packages, making it a flexible choice for managing project requirements.

Using Conda allows you to create isolated environments for different projects, preventing version conflicts between dependencies and ensuring reproducibility. This is crucial for maintaining a stable development environment across different projects and teams.

Link to YouTube tutorial:

Ruff for code linting

Ruff is a fast Python linter designed to catch potential issues and enforce coding standards. It helps maintain code quality by identifying problems early in the development process.

Linting with Ruff helps enforce coding standards, catch syntax errors, and improve code readability. It reduces the likelihood of bugs and ensures that the code adheres to best practices.

Link to YouTube tutorial:

Pytest for testing

Pytest is a popular testing framework for Python that provides simple and scalable test writing. It supports fixtures, parameterized testing, and various plugins for extending its functionality.

Testing is crucial for ensuring code reliability and catching bugs early. Pytest's flexible and powerful features help you write and manage tests efficiently, contributing to a stable and maintainable codebase.

Link to YouTube tutorial:

Pre-commit hooks

Pre-commit hooks are scripts that run automatically before a commit is finalized. They can perform tasks like code formatting, linting, and validation, ensuring code quality before changes are committed to the repository.

Pre-commit hooks automate code quality checks, reducing the risk of introducing errors into the codebase. They help maintain consistency and enforce standards across the development team.

Link to YouTube tutorial:

GitHub for version control

GitHub is a widely used platform for hosting and managing code repositories. It provides version control, collaboration tools, and integration with various CI/CD services.

Version control with GitHub allows for tracking changes, collaboration with other developers, and managing different versions of the project. It enhances team collaboration and code management.

Link to YouTube tutorial:

Code cleanliness

Maintaining clean code is an essential skill. Clean code is not just about writing code that works; it's about writing code that is readable, maintainable, and scalable. We'll focus on best practices for keeping your Python code clean and well-organized, ensuring that it remains robust and easy to work with over time. Note that these are not extensive

Meaningful naming

Use descriptive names

Names should clearly convey the purpose of the variables, functions, and classes. Avoid generic names like `data` or `temp` unless their context is obvious.

Example:

```
# Bad naming
def func(x):
    return x * 2

# Good naming
def double_value(number):
    return number * 2
```

Use consistent naming conventions

Follow consistent naming conventions to improve readability. In Python, the conventions are:

- **Variables and functions**: Use `snake_case` (e.g., `total_amount`, `calculate_sum`).
- **Classes**: Use `CamelCase` (e.g., `InvoiceManager`, `DataProcessor`).
- **Constants**: Use `UPPER_CASE` (e.g., `MAX_RETRIES`, `DEFAULT_TIMEOUT`).

Small and focused functions

Single responsibility principle

Each function should do one thing and do it well. Break down complex functions into smaller, more manageable ones.

Example:

```
# Bad: Single function with multiple responsibilities
def process_order(order):
    validate_order(order)
    calculate_total(order)
    save_order_to_database(order)

# Good: Functions with a single responsibility
def validate_order(order):
    # validation logic
    pass

def calculate_total(order):
    # total calculation logic
    pass

def save_order_to_database(order):
    # save logic
    pass

def process_order(order):
    validate_order(order)
    calculate_total(order)
    save_order_to_database(order)
```

Avoid deep nesting

Deeply nested code can be hard to follow. Use early returns to reduce nesting levels.

Example:

```
# Bad: Deep nesting
def process_data(data):
    if data is not None:
```

```
        if len(data) > 0:
            if isinstance(data, list):
                # process the list
                pass

# Good: Reduced nesting
def process_data(data):
    if data is None or len(data) == 0 or not isinstance(data, list):
        return

    # process the list
    pass
```

Clear and consistent formatting

Follow PEP 8

Adhere to PEP 8, the Python style guide, to ensure consistent formatting. Key guidelines include:

- **Indentation**: Use 4 spaces per indentation level.
- **Line Length**: Limit all lines to a maximum of 79 characters.
- **Blank Lines**: Use blank lines to separate functions, classes, and sections within functions.

To make things easy, we will use ruff as a standard in our lab.

Example:

```
# PEP 8 compliant code
def calculate_area(radius):
    import math
    return math.pi * (radius ** 2)

class Circle:
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return calculate_area(self.radius)
```

Use docstrings

Use docstrings to document your modules, classes, and functions. Docstrings should describe the purpose, parameters, and return values.

Example:

```
def calculate_area(radius):
    """
    Calculate the area of a circle given its radius.

    Args:
        radius (float): The radius of the circle.

    Returns:
        float: The area of the circle.
    """
    import math
    return math.pi * (radius ** 2)
```

Error handling

Use exceptions appropriately

Handle exceptions gracefully and use specific exception types to avoid catching unexpected errors.

Example:

```
# Bad: Catching all exceptions
try:
    result = 10 / 0
except Exception as e:
    print(f"An error occurred: {e}")

# Good: Catching specific exceptions
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print(f"Cannot divide by zero: {e}")
```

Avoid empty `except` clauses

Empty `except` clauses can hide errors and make debugging difficult. Always handle exceptions with proper logging or user feedback.

Example:

```
# Bad: Empty except clause
try:
    # risky operation
    pass
except:
    pass

# Good: Proper exception handling
try:
    # risky operation
    pass
except ValueError as e:
    print(f"ValueError occurred: {e}")
```

Refactoring

Regularly review and refactor code to improve structure and readability. Refactoring can include renaming variables, breaking down functions, or reorganizing code.

Keeping your code tidy ensures it is readable and maintainable. Clean code is easier to understand, debug, and enhance. Here are some tips for maintaining code cleanliness:

For more detailed style guide, we will use Google Python Style Guide.

Decoupled code

In a research lab, Python code often forms the backbone of data analysis, simulation, and experimentation. However, as research projects grow in complexity and involve multiple researchers, writing well-structured and decoupled code becomes crucial. Decoupled code—code that is modular and has minimal interdependencies—enhances readability, maintainability, and testability, ultimately leading to more robust and scalable research tools. This chapter explores the principles of writing decoupled Python code and offers practical guidance tailored to the needs of a research lab environment.

Understanding decoupling in code

Decoupling refers to the practice of reducing dependencies between different parts of a program. When code is decoupled, changes in one part of the code have minimal impact on others. This is particularly important in research labs where:

- Different team members might work on different aspects of the codebase.
- Research projects often evolve, requiring frequent changes and updates to the code.
- Code might need to interface with various tools, libraries, and datasets.

The primary benefits of decoupling are:

1. **Modularity**: Breaking down the code into distinct modules or components.
2. **Testability**: Isolated components are easier to test independently.
3. **Maintainability**: Changes in one module have less chance of breaking other parts of the code.
4. **Reusability**: Modular components can be reused across different projects or experiments.

Principles of decoupled Python code

Single responsibility principle (SRP)

Each module or class should have only one reason to change, meaning it should have one primary responsibility. For instance, a module handling data cleaning should not be involved in data visualization.

Example: Suppose you have a module named `data_processing.py`. It should focus solely on data cleaning and preprocessing, while another module, `data_visualization.py`, should handle plotting and generating graphs.

```
# data_processing.py
def clean_data(data):
    # Implementation of data cleaning
    pass

def preprocess_data(data):
    # Implementation of data preprocessing
    pass

# data_visualization.py
import matplotlib.pyplot as plt

def plot_data(data):
    plt.plot(data)
    plt.show()
```

Dependency injection

Avoid hard-coding dependencies within your functions or classes. Instead, pass dependencies as arguments. This approach makes your code more flexible and easier to test.

Example: Instead of hardcoding a data source, pass it as a parameter.

```
# Instead of this
def analyze_data():
    data = load_data_from_file('data.csv')
    # Process data

# Use dependency injection
def analyze_data(data_loader):
    data = data_loader()
    # Process data

# Example data loader function
def load_data_from_file():
    return read_csv('data.csv')
```

Use interfaces and abstract classes

Define abstract classes or interfaces to specify the methods that concrete implementations should provide. This approach helps in decoupling the code from specific implementations and allows easier replacement or modification of components.

Example:

```
from abc import ABC, abstractmethod

class DataLoader(ABC):
    @abstractmethod
    def load(self):
        pass

class CSVDataLoader(DataLoader):
    def load(self):
        # Load data from CSV
        pass

class JSONDataLoader(DataLoader):
    def load(self):
        # Load data from JSON
        pass
```

Separation of concerns

Ensure that different aspects of your code are managed separately. For example, keep data handling, computation, and presentation concerns distinct from each other.

Example:

```
# data_manager.py
def load_data(filename):
    # Code to load data
    pass

# analysis_engine.py
def perform_analysis(data):
    # Code to analyze data
    pass
```

```python
# report_generator.py
def generate_report(results):
    # Code to generate report
    pass
```

Conclusion

Decoupled Python code is essential for maintaining clarity, flexibility, and reliability in a research lab setting. By adhering to principles such as the Single Responsibility Principle, dependency injection, and separation of concerns, researchers can create codebases that are easier to understand, test, and modify. Implementing these practices ensures that research software remains adaptable and robust, supporting the dynamic needs of scientific inquiry.

For more information on writing decoupled code, explore here.

Code testing

In the research lab environment, testing Python code is crucial for ensuring accuracy, reliability, and reproducibility of scientific results. `pytest` is one of the most popular testing frameworks in Python due to its simplicity, flexibility, and powerful features.

Basic test structure

`pytest` automatically discovers and runs tests based on their naming conventions. Tests should be placed in files named `test_*.py` or `*_test.py`, and test functions should start with `test_`.

Example:

```
# test_calculator.py
def add(x, y):
    return x + y

def test_add():
    assert add(1, 2) == 3
    assert add(-1, 1) == 0
```

Run tests using the `pytest` command:

```
pytest
```

Writing tests with `pytest`

Basic assertions

Use `assert` statements to check if the code behaves as expected. `pytest` will report failed assertions with detailed information.

Example:

```
def multiply(x, y):
    return x * y

def test_multiply():
    assert multiply(2, 3) == 6
    assert multiply(0, 5) == 0
```

Using fixtures

Fixtures are used to set up and tear down resources needed for tests. They are defined using the `@pytest.fixture` decorator and can be scoped to functions, classes, modules, or sessions.

Example:

```
import pytest

@pytest.fixture
def sample_data():
    return [1, 2, 3, 4, 5]

def test_sum(sample_data):
    assert sum(sample_data) == 15
```

Parameterized tests

Parameterized tests allow you to run the same test function with multiple sets of inputs using the `@pytest.mark.parametrize` decorator.

Example:

```
import pytest

@pytest.mark.parametrize("a, b, expected", [
    (1, 2, 3),
    (-1, 1, 0),
    (2, 2, 4),
])
def test_add(a, b, expected):
    assert add(a, b) == expected
```

Testing for exceptions

You can check for expected exceptions using the `pytest.raises` context manager.

Example:

```
import pytest

def divide(x, y):
    if y == 0:
        raise ValueError("Cannot divide by zero")
    return x / y

def test_divide():
    with pytest.raises(ValueError):
        divide(1, 0)
```

Customizing test output

`pytest` provides options for customizing test output, such as verbosity levels and formatting.

Example:

Run tests with verbose output:

```
pytest -v
```

Generate a test report in a JUnit-compatible format:

```
pytest --junitxml=report.xml
```

Advanced features of `pytest`

Plugins

`pytest` supports a wide range of plugins to enhance its functionality. Some popular plugins include:

- **`pytest-cov`**: Provides code coverage reporting.
- **`pytest-mock`**: Simplifies mocking of objects and functions.
- **`pytest-xdist`**: Allows parallel test execution and distributed testing.

Install and use plugins via pip:

```
pip install pytest-cov pytest-mock
```

Example with `pytest-cov`:

```
pytest --cov=my_module
```

Fixtures with scope and autouse

Control the scope and automatic application of fixtures using the `scope` and `autouse` parameters.

Example:

```
import pytest

@pytest.fixture(scope="module", autouse=True)
def setup_module():
    print("\nSetting up module...")
    yield
    print("\nTearing down module...")
```

Integrating `pytest` into the research workflow

Continuous integration (CI)

Integrate `pytest` with CI/CD pipelines to automatically run tests on code changes. Tools like GitHub Actions, GitLab CI, and Jenkins support `pytest` integration.

Example with GitHub Actions:

Create a `.github/workflows/test.yml` file:

```
name: Run Tests

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
```

```
    steps:
      - uses: actions/checkout@v2
      - name: Set up Python
        uses: actions/setup-python@v2
        with:
          python-version: '3.8'
      - name: Install dependencies
        run: |
          pip install pytest
      - name: Run tests
        run: |
          pytest
```

Code coverage

Use `pytest-cov` to measure test coverage and ensure that critical parts of your code are tested.

Example:

```
pytest --cov=my_module --cov-report=html
```

Test documentation

Document your tests to explain their purpose and expected outcomes, which helps new team members understand the testing strategy and rationale.

Conclusion

`pytest` is a powerful and flexible testing framework that enhances the reliability and maintainability of Python code in a research lab setting. By leveraging `pytest`'s features such as fixtures, parameterized tests, and plugins, researchers can create comprehensive test suites that ensure code quality and facilitate collaboration. Integrating `pytest` into your research workflow, including continuous integration and code coverage, will help maintain the integrity of your codebase and support reproducible research.

By adopting these practices, you can build a robust testing framework that contributes to more reliable and effective scientific research.

Learn more about testing here.

Code documentation

Effective documentation is essential in a research lab environment, where Python code plays a critical role in data analysis, simulations, and various computational tasks. Well-documented code not only aids in understanding and maintaining the codebase but also facilitates collaboration among researchers and ensures the reproducibility of results. This chapter provides a comprehensive guide on documenting Python code, focusing on practices and tools that enhance clarity and usability in a research context.

Importance of documentation

Documentation serves several crucial purposes:

1. **Clarity**: Helps researchers understand the code's purpose and functionality.
2. **Maintenance**: Facilitates easier updates and debugging by providing insights into the code's structure and logic.
3. **Collaboration**: Ensures that team members can work effectively on shared codebases.
4. **Reproducibility**: Documents the steps and methods used in analysis, enabling others to replicate experiments.

Types of documentation

Effective documentation covers various aspects of a codebase:

Code comments

Comments within the code explain specific lines or sections of code, making it easier to understand the logic and intentions of the code.

- **Inline Comments**: Provide explanations for individual lines of code.
- **Block Comments**: Explain larger sections or functions.

Example:

```
def calculate_statistics(data):
    """
    Calculate and return basic statistics for the given data.

    Args:
        data (list): A list of numerical values.

    Returns:
        tuple: A tuple containing the mean and standard deviation of the data.
    """
    mean = sum(data) / len(data)  # Calculate mean
    variance = sum((x - mean) ** 2 for x in data) / len(data)  # Calculate variance
    std_dev = variance ** 0.5  # Calculate standard deviation
    return mean, std_dev
```

Docstrings

Docstrings are used to describe the purpose, parameters, and return values of functions, classes, and modules. They are essential for generating documentation automatically and integrating with IDEs and documentation tools.

- **Function Docstrings**: Explain the purpose, arguments, and return values of a function.
- **Class Docstrings**: Describe the class's purpose, attributes, and methods.
- **Module Docstrings**: Provide an overview of the module and its contents.

Example:

```
class DataAnalyzer:
    """
    A class for analyzing and processing data.

    Attributes:
        data (list): The data to be analyzed.

    Methods:
        calculate_mean(): Returns the mean of the data.
        calculate_median(): Returns the median of the data.
    """

    def __init__(self, data):
```

```
        """
        Initialize the DataAnalyzer with data.

        Args:
            data (list): The data to be analyzed.
        """
        self.data = data

    def calculate_mean(self):
        """
        Calculate and return the mean of the data.

        Returns:
            float: The mean of the data.
        """
        return sum(self.data) / len(self.data)
```

README files

README files provide an overview of the project, including its purpose, installation instructions, usage guidelines, and contribution details. They are often the first point of contact for users and contributors.

Example:

```
# Data Analysis Toolkit

# Overview
This toolkit provides various utilities for analyzing and processing data.

# Installation
To install the toolkit, run:
```bash
pip install data-analysis-toolkit
```
```

Usage

To analyze data, use the following commands:

```
from data_analysis_toolkit import DataAnalyzer

data = [1, 2, 3, 4, 5]
analyzer = DataAnalyzer(data)
mean = analyzer.calculate_mean()
print(mean)
```

Tools for generating documentation

Sphinx

Sphinx is a documentation generator that creates HTML, PDF, and other formats from reStructuredText files. It is often used for creating comprehensive project documentation.

- **Installation**:

```
pip install sphinx
```

- **Usage**:

Create a Sphinx documentation directory:

```
sphinx-quickstart
```

Generate documentation:

```
sphinx-build -b html source build
```

Best practices for documentation

Write clear and concise descriptions

Ensure that descriptions are clear and avoid jargon. Provide enough detail to understand the purpose and functionality of the code without being overly verbose.

Update documentation regularly

Keep documentation up-to-date with code changes. Outdated documentation can lead to confusion and errors.

Use consistent style

Follow consistent formatting and style guidelines throughout the documentation. This includes using the same terminology, formatting conventions, and level of detail.

Include Examples

Provide examples to demonstrate how to use functions, classes, or modules. Examples help users understand practical applications of the code.

Document edge cases and limitations

Highlight any edge cases, limitations, or known issues in the documentation. This prepares users for potential problems and guides them in handling exceptions.

Conclusion

Documenting Python code in a research lab is crucial for maintaining clarity, facilitating collaboration, and ensuring reproducibility. By utilizing code comments, docstrings, README files, and inline documentation, researchers can create comprehensive and useful documentation. Tools like Sphinx, MkDocs, and pdoc can automate and enhance the documentation process.

Adhering to best practices for writing clear and concise documentation will support effective communication and collaboration within the research team and help ensure that your code is usable and maintainable over time. By integrating these practices, you contribute to a well-organized and reliable research environment, fostering better understanding and reproducibility of scientific work.

Project structure

At iHuman lab, we follow the project structure (shown below) which serves as a flexible guide rather than a strict mandate. It is designed to provide a clear framework for organizing our research endeavors effectively. The structure encompasses essential components such as documentation in the `docs/` directory, configuration files in `config/`, and a dedicated `data/` directory for managing datasets. The `notebooks/` directory houses Jupyter notebooks for exploratory analysis, while the `src/` directory contains modularized source code for data processing, feature engineering, model development, visualization, and utility functions. Unit tests are stored in `tests/` to ensure the reliability of our implementations. While this structure offers a solid foundation, it remains adaptable to meet the specific needs and nuances of each research project we undertake, allowing for customization and refinement as required.

```
research_project_name/

  README.md                          # Project overview and instructions
  docs/                              # Documentation

  config/                            # Configuration files
      config.yaml                   # Configuration file for parameters
      logging.yaml                  # Configuration file for logging

  data/                              # Directory for datasets
      raw/                          # Raw data files (immutable)
      processed/                    # Processed data files (generated)
      interim/                      # Intermediate data files (temporary)

  notebooks/                         # Jupyter notebooks for exploration
      exploratory_analysis.ipynb

  src/                               # Source code for the project
      __init__.py
      main.py                       # Main script to run the project
      data/                         # Module for data processing
          __init__.py
          preprocess.py            # Preprocessing functions
          load_data.py             # Data loading functions
```

```
    dataset/                    # Module for creating datasets
        __init__.py
        create_dataset.py       # Functions to create datasets
    features/                   # Module for creating features
        __init__.py
        feature_engineering.py  # Feature engineering functions
    models/                     # Module for defining models
        __init__.py
        model.py                # Model architecture and training functions
    visualization/              # Visualization module
        __init__.py
        visualize.py            # Visualization functions
    utils.py                    # Utility functions used across modules

tests/                          # Unit tests

environment.yml                 # Conda env file specifying dependencies
```

To facilitate the easy creation and adoption of this project structure across our research lab, we have opted to use iHuman Lab's Cookiecutter template.

To use the template

```
$ pip install cookiecutter
```

or

```
$ conda config --add channels conda-forge
$ conda install cookiecutter
```

To start a new project, run:

```
$ cookiecutter https://github.com/iHuman-Lab/cookiecutter-data-science.git
```

Using Git

GitHub is a powerful platform for version control and collaboration, allowing developers to work together on projects from anywhere in the world. It utilizes Git, a widely used version control system, to track changes in code and manage projects effectively. This tutorial will guide you through the basics of using GitHub, from setting up your account and installing Git to creating repositories and collaborating with others. Whether you're a beginner or looking to refresh your skills, this guide will help you get started on both Windows and Linux.

Here is a very good detailed tutorial if you want to skip reading the rest of the tutorial.

Create a GitHub Account

- Go to GitHub.
- Click on "Sign up" in the top right corner.
- Fill in the required information (username, email, password) and follow the prompts to create your account.
- Check your email inbox for a verification email from GitHub and follow the instructions to verify your account.

Set up Git

For Windows Users:

- Download the Git installer for Windows from git-scm.com.
- Run the installer and follow the setup instructions, accepting the default options unless you have specific preferences.

For Linux Users:

- Open a terminal.
- Install Git using your package manager. For example, on Ubuntu, you can run:

```
sudo apt update
sudo apt install git
```

Configure Git

- Linux users use Terminal and Windows users use Powershell.
- Set your username and email:

```
git config --global user.name "Your Name"
git config --global user.email "your_email@example.com"
```

Create a New Repository

- Log into GitHub and click on the "+" icon in the top right corner.
- Select "New repository."
- Fill in the repository name, description (optional), and choose to make it public or private.
- Click "Create repository."

Set Up a Local Repository

- Open a terminal/powershell and navigate to the directory where you want to create your project:

```
cd /path/to/your/directory
```

- Create a new directory for your project:

```
mkdir my-project
cd my-project
```

- Initialize a Git repository:

```
git init
```

Add Files and Make Your First Commit

- Create a new file (e.g., `README.md`) using a text editor or the terminal:

```
echo "# My Project" >> README.md
```

- Add the file to the staging area:

```
git add README.md
```

- Commit the changes with a message:

```
git commit -m "Initial commit"
```

Connect Local Repository to GitHub

- Link your local repository to the GitHub repository:

```
git remote add origin https://github.com/your_username/my-project.git
```

- Push your commits to GitHub:

```
git push -u origin master
```

Making Changes and Collaborating

- Edit files using a text editor of your choice (Notepad, Visual Studio Code, Vim, Nano, etc.).
- Stage and commit your changes:

```
git add .
git commit -m "Updated the README"
```

- Push your changes to GitHub:

```
git push
```

- To collaborate with others, you can fork repositories, create branches, and submit pull requests:
  - **Fork a Repository**: Click the "Fork" button on a repository to create your own copy.
  - **Create a Branch**:

    ```
    git checkout -b new-feature
    ```

  - **Submit a Pull Request**: After pushing your branch, go to the original repository on GitHub and create a pull request.

Managing Your Repository

- Go to your repository on GitHub to see your files, commits, branches, and issues.
- Create issues to track tasks or bugs.
- Use the Projects feature to organize and prioritize your work.
- Automate workflows using GitHub Actions to build, test, and deploy your projects.

Part III

Tutorials

Python code structure

Overview

In this tutorial, we will cover the following concepts related to organizing and using Python code:

1. **Modules**
2. **Packages**
3. **Subpackages**
4. **Importing and Using Modules**
5. **Understanding `__init__.py`**
6. **Running the Code**

Project structure

Before diving into the details, here's the overall structure of the project we will create:

```
project/
    math_operations.py
    utility.py
    package/
        __init__.py
        module_a.py
        module_b.py
        subpkg/
            __init__.py
            module_c.py
    main.py
```

---

1. Modules

A **module** is a single Python file that can contain functions, classes, and variables.

Creating a module

Create a file named `math_operations.py`:

```
# math_operations.py

def square(number):
    """Returns the square of a number."""
    return number ** 2

def add(a, b):
    """Returns the sum of two numbers."""
    return a + b
```

---

2. Packages

A **package** is a directory that contains an `__init__.py` file and can include modules or subpackages.

Creating a package

1. Create a directory named `package`.
2. Inside `package`, create:
    - `__init__.py`
    - `module_a.py`
    - `module_b.py`

Structure:

```
package/
    __init__.py
    module_a.py
    module_b.py
```

`module_a.py`:

```python
# package/module_a.py

def double(number):
    """Returns double the input number."""
    return number * 2

def find_max(numbers):
    """Returns the maximum number from a list."""
    return max(numbers) if numbers else None
```

module_b.py:

```python
# package/module_b.py

from .module_a import double  # Relative import

def multiply(a, b):
    """Returns the product of two numbers."""
    return a * b

def subtract(a, b):
    """Returns the difference between two numbers."""
    return a - b

def double_and_subtract(a, b):
    """Uses double from module_a and then subtracts."""
    return subtract(double(a), b)
```

---

3. Subpackages

A **subpackage** is a package inside another package.

Creating a subpackage

1. Inside `package`, create a directory named `subpkg`.
2. Inside `subpkg`, create:
    - `__init__.py`

- module_c.py

Structure:

```
package/
    subpkg/
        __init__.py
        module_c.py
```

module_c.py:

```
# package/subpkg/module_c.py

from ..module_a import double  # Relative import

def sum_numbers(numbers):
    """Returns the sum of a list of numbers."""
    return sum(numbers)
```

__init__.py:

To make `sum_numbers` accessible when importing the subpackage:

```
# package/subpkg/__init__.py

from .module_c import sum_numbers
```

---

4. Importing and using modules

Example main script

Create a file named `main.py` in the root directory of your project:

```
# main.py

from math_operations import square, add
from utility import cube
from package import double, find_max, multiply, subtract
```

```python
from package.subpkg import sum_numbers

if __name__ == "__main__":
    num = 4
    print(f"Square of {num}: {square(num)}")
    print(f"Add 2 and 3: {add(2, 3)}")
    print(f"Cube of {num}: {cube(num)}")
    print(f"Double of {num}: {double(num)}")
    print(f"Multiply: {multiply(5, 3)}")
    print(f"Subtract: {subtract(5, 3)}")

    numbers = [1, 2, 3, 4]
    print(f"Sum of numbers: {sum_numbers(numbers)}")
    print(f"Double and Subtract: {double_and_subtract(5, 3)}")
```

Importing within the same directory

You can import functions or classes from another module within the same directory using the `import` statement.

Creating `utility.py`:

Add another module named `utility.py` in the root directory:

```python
# utility.py

def cube(number):
    """Returns the cube of a number."""
    return number ** 3
```

Relative imports

Relative imports allow you to import modules or packages relative to the current module's location within the package hierarchy. This is useful for maintaining clear imports and avoiding conflicts.

For example, in `module_b.py`, you can see the line:

```python
from .module_a import double
```

This is a relative import, indicating that `module_a` is in the same package as `module_b`.

In `module_c.py`, the line:

```
from ..module_a import double  # Importing double from module_a in the parent package
```

shows a relative import from the parent package.

---

5. Understanding `__init__.py`

The `__init__.py` file is essential for defining packages in Python.

Importance of `__init__.py`

1. **Package Initialization**: Indicates that the directory should be treated as a Python package.
2. **Namespace Management**: Helps to define the package namespace.
3. **Control Over Imports**: You can control which modules are accessible when importing the package.
4. **Initialization Logic**: Allows you to include setup or initialization code.

Example of `__init__.py`

Here's a simple example illustrating how `__init__.py` works:

Directory Structure:

```
package/
    __init__.py
    module_a.py
    module_b.py
```

Content of `module_a.py`:

```
# module_a.py

def get_number():
    """Returns a fixed number."""
    return 42
```

Content of `module_b.py`:

```
# module_b.py

def get_text():
    """Returns a greeting."""
    return "Hello, World!"
```

Content of `__init__.py`:

```
# __init__.py

from .module_a import get_number
from .module_b import get_text
```

Using the package

```
from package import get_number, get_text

print(get_number())  # Outputs: 42
print(get_text())    # Outputs: Hello, World!
```

---

6. Running the code

To run your project, navigate to the directory containing `main.py` and execute:

```
python main.py
```

Expected output

When you run the code, you should see the following output:

```
Square of 4: 16
Add 2 and 3: 5
Cube of 4: 64
Double of 4: 8
Multiply: 15
```

```
Subtract: 2
Sum of numbers: 10
Double and Subtract: 4
```

Python resources

These are some of the resource to introduce you to basics Python. The resources contain videos, articles, books. Please feel free to come to iHuman lab and ask anyone at lab to get more information.

General resources

Getting started

Whether you're a beginner looking to set up your first Python environment or an experienced developer aiming to enhance efficiency, this tutorial guides you through configuring tools, managing dependencies, and mastering workflows that maximize your productivity.

Real python: Perfect Your Python Development Setup

- Explore the essential steps to streamline and optimize your Python development environment
- Dive into foundational concepts, discover expert tips, and learn to tailor your development setup to suit your specific needs

Conda

Dive deeper into efficient project management with Conda, a versatile tool for creating and managing Python environments. Whether you're developing software or conducting research, Conda ensures seamless environment setup, eliminating dependency conflicts and promoting project reproducibility. Navigate Conda confidently to streamline your coding workflow and enhance collaboration.

Discover Conda's capabilities: Why You Need Python Environments and How to Manage Them with Conda

- Master the installation and utilization of Conda to create isolated Python environments tailored to your project requirements.
- Learn best practices for environment management to optimize development efficiency and maintain code integrity.

Scientific computing

Elevate your scientific computing endeavors with "Good Enough Practices in Scientific Computing." This comprehensive guide equips you with essential methodologies for optimizing code performance, ensuring data integrity, and promoting reproducibility in research settings. Adopt best practices to enhance the efficiency and transparency of your scientific coding workflows.

Enhance your scientific practices here: Good Enough Practices in Scientific Computing

- Navigate the guide to implement foundational practices in scientific programming, including effective data management and workflow optimization.
- Incorporate version control strategies and virtual environments to maintain project integrity and facilitate collaboration in research environments.

Open source

Engage with the dynamic world of open-source development through the Open Source Guide. Embrace collaboration and community-driven innovation as you navigate the principles of contributing effectively to open-source projects. From mastering version control with Git to exploring diverse projects aligned with your interests, empower yourself to make meaningful contributions and foster professional growth.

Join the open-source community here: Open Source Guide

- Familiarize yourself with the ethos and methodologies of open-source development outlined in the Open Source Guide.
- Enhance your proficiency in Git version control to efficiently manage code revisions and facilitate collaborative coding efforts.

PyVideo

Immerse yourself in PyVideo's repository of educational content, where industry experts share profound insights and innovative practices in Python programming. From foundational concepts to advanced techniques and emerging trends, PyVideo enriches your learning journey with comprehensive tutorials and conference talks. Stay informed, inspired, and ahead in the ever-evolving Python ecosystem.

Explore PyVideo's resources here: PyVideo

- Navigate PyVideo to access a wealth of video content covering diverse topics in Python programming.
- Take notes and integrate key learnings from expert presentations into your coding practices and project development.

WTF Python

Delve into the intriguing intricacies of Python with WTF Python—a curated collection of code snippets that unravel surprising behaviors and lesser-known features. Challenge yourself to decode these puzzling aspects of Python programming, gaining deeper insights into its flexibility and functionality. Discover the nuances that make Python a preferred language for diverse applications.

Unveil Python's mysteries here: WTF Python

- Engage with WTF Python to explore unconventional Python behaviors and expand your programming horizons.
- Experiment with provided code snippets to deepen your understanding of Python's unique capabilities and edge cases.

Python Tutor

Python Tutor, a sophisticated platform where code transcends the screen. Visualize your code execution step-by-step, gaining invaluable insights into Python's mechanics. This interactive tool transforms abstract concepts into tangible understanding, making complex programming principles accessible and engaging.

Explore Python Tutor here: Python Tutor

- Utilize Python Tutor to visualize code execution and understand variable interactions dynamically.
- Experiment with different code snippets to solidify your grasp on fundamental Python concepts.

Visualizing NumPy

Capitalize on NumPy's prowess in numerical computing and data manipulation through interactive tutorials offered by Visualizing NumPy Operations. Enhance your proficiency in array operations, indexing techniques, and vectorized computations with visual demonstrations that illustrate complex concepts effectively. Strengthen your skills to tackle sophisticated data-centric challenges with confidence and precision.

Master NumPy's capabilities here: Visualizing NumPy Operations

- Explore interactive tutorials on Visualizing NumPy Operations to deepen your understanding of NumPy's capabilities and applications.
- Apply learned concepts to practical scenarios, refining your data manipulation skills and optimizing performance in computational tasks.

Websites

Are you gearing up for job interviews or seeking internship opportunities in the realm of programming? Mastering Python can significantly enhance your skillset and boost your competitiveness in the job market. Here's a tailored selection of platforms designed to equip you with essential Python skills and prepare you effectively for technical assessments:

- Edx If you are new to python programming, definitely take Introduction to Computer Science and Programming Using Python course. It is one of the best courses out there from which you can learn lot of fundamental python programming skills.
- HackerRank HackerRank has lot of resources to learn many programming languages. Regestration is free.
- LeetCode LeetCode is much intensive platform to learn coding. It can be used if you want to prepare for interviews.

PyCon

PyCon is one of the best way to get in touch with great Python community. Lot of great talks and tutorials (highly recommended!).

Pytorch

As a new student, Pytroch will be an essential tool for your deep learning research due to its flexibility and extensive community support. To streamline your workflow and boost productivity, we've compiled a guide to key PyTorch libraries and tools that simplify model development, training, and evaluation.

A github repository with lot of resources: The incredible Pytorch

PyTorch Lightning

GitHub Link: PyTorch Lightning

PyTorch Lightning is our preferred framework for developing deep learning models in the lab. It automates the tedious aspects of deep learning research, such as setting up training loops, logging metrics, and handling distributed training across multiple GPUs or TPUs. By standardizing best practices and providing a clean interface, PyTorch Lightning enables rapid prototyping and seamless scaling of research projects.

Key Features:

- **Automated Training Loop**: Write minimal code to define your model, optimizer, and data loaders, and PyTorch Lightning takes care of the rest.
- **Reproducibility**: Ensures deterministic training behavior for reproducible research results.
- **Scalability**: Easily scale your models from single GPU to multi-GPU or even distributed training without changing your codebase.
- **Experiment Management**: Built-in support for logging, visualization, and tracking experiments, making it easier to monitor and compare model performance.

In our lab, we predominantly use PyTorch Lightning for its ability to accelerate the development cycle and ensure consistency across experiments. It encourages best practices in deep learning research and facilitates collaboration among team members.

Other PyTorch related links

- https://github.com/szymonmaszke/torchfunc
- https://github.com/wkentaro/pytorch-for-numpy-users
- https://github.com/suriyadeepan/torchtest

Ray

Another python package we extensively use in our lab is Ray. Ray a powerful parallel processing library designed to streamline distributed computing tasks in Python. Whether you're working with machine learning models, simulations, or data processing pipelines, Ray provides efficient tools for scaling your applications across multiple cores or clusters.

Key features of Ray

Task parallelism with remote functions

Ray enables you to define functions (`@ray.remote`) that can execute asynchronously across multiple workers. These functions can leverage the computing resources efficiently, executing tasks concurrently without blocking the main thread.

Actor model for stateful services

Ray's Actor model allows you to create and manage stateful objects that maintain their internal state across multiple function invocations. This is useful for building complex applications where maintaining state across distributed components is crucial.

Distributed data processing with Ray libraries

Ray comes with additional libraries that extend its capabilities for distributed computing:

- **Ray Tune**: Scalable hyperparameter tuning. Ray Tune Documentation
- **Ray RLlib**: Scalable reinforcement learning. Ray RLlib Documentation
- **Ray Serve**: Scalable and efficient serving of machine learning models. Ray Serve Documentation

Resources and documentation

Explore more about Ray's capabilities, including installation guides, tutorials, and API references, through its official documentation:

- Ray GitHub Repository
- Ray Documentation

BCI resources

These are some of the resource to introduce you to basics of brain computer interfaces (BCI). The resources contain videos, articles, books. Please feel free to come to iHuman lab and ask anyone at lab to get more information.

Videos

Swartz Center for Computational Neuroscience

- **Introduction to BCI and EEG**
  - Start with the initial videos (video-1 to video-7) to grasp the fundamentals of BCI and EEG.
  - These videos provide a solid foundation for understanding how brain activity can be used to control external devices.
  - Watch the videos here

> Tip
>
> These introductory videos are highly recommended as they cover the basics you need to get started.

Mike X Cohen

- **General Neural Time-Series Analysis**
  - Explore advanced concepts in neural time-series analysis, focusing on methods to analyze and interpret brain signals over time.
  - The videos primarily use Matlab, but you can replicate the analysis in Python using the mne package.
  - Watch the videos here

> **Tip**
>
> Adapting Matlab scripts to Python will not only deepen your understanding of neural data analysis but also enhance your proficiency in Python programming.

GitHub links

- **NeuroTechX/awesome-bci**
  - This repository is a curated collection of resources, tools, and projects related to BCI research.
  - Explore various software tools, datasets, and community-driven projects that can aid your research.
  - Visit the repository

Books

- **Brain-Computer Interfacing: An Introduction** by Rajesh P.N. Rao
  - This book provides a comprehensive introduction to BCI technology, covering its principles, applications, and future trends.
  - Access the book
- **Analyzing Neural Time Series Data: Theory and Practice** by Mike X. Cohen
  - Learn detailed methodologies and practical techniques for analyzing neural time-series data.
  - Gain insights into advanced signal processing techniques applicable to BCI research.
  - Explore the book

MNE Python

Another important package, we use extensively in our lab processing the physiological data is MNE-Python. MNE-Python is a Python package specifically designed for the analysis of brain signals, collected using techniques like magnetoencephalography (MEG) and electroencephalography (EEG). It provides a robust set of tools for processing, analyzing, and visualizing neural data, making it a preferred choice for researchers and clinicians in the field of neuroscience.

Key features of MNE-Python include:

- **Data Handling:** Import and manipulate data from various formats such as .fif, .edf, and .bdf.
- **Preprocessing:** Remove noise, filter data, and interpolate bad channels.

- **Analysis:** Perform time-domain and frequency-domain analyses, including time-frequency decomposition.
- **Visualization:** Plot data in 2D and 3D to visualize sensor and source-space information.
- **Source Localization:** Estimate the location of neural sources responsible for recorded signals.
- **Statistics:** Conduct statistical tests to analyze differences in brain activity.

MNE-Python supports a wide range of functionalities tailored to both novice users and advanced researchers, facilitating comprehensive exploration and interpretation of neuroimaging data.

For more information and detailed documentation, visit MNE-Python website.

Other links

- Video tutorials on MNE python
- Official MNE tutorials

> **Task**
>
> 1. Explore basic EEG processing tutorials here: EEG Tutorials
> 2. For more detailed tutorials go through the tutorials and implement standard functions using the EEG data in MNE website. Reach out to Hemanth to get the EEG data.

Lab Streaming Layer (LSL)

Lab streaming layer (LSL) is an open-source networked middleware ecosystem to stream, receive, synchronize, and record neural, physiological, and behavioral data streams acquired from diverse sensor hardware.

It reduces complexity and barriers to entry for researchers, sensor manufacturers, and users through a simple, interoperable, standardized API to connect data consumers to data producers while abstracting obstacles such as platform differences, stream discovery, synchronization and fault-tolerance.

We use LSL as a standard platform to interface multiple sensor and simulation softwares in our lab.

- **Overview and Importance**
  - LSL is a crucial middleware platform used in our lab to synchronize and record data from various sensors.
  - It simplifies data integration across different hardware and software platforms, ensuring seamless connectivity and interoperability.

- Learn more about LSL
- Learn about pylsl

EEG resources

One of the main modalities we use in our lab to study Human-robot interaction is EEG (others include EMG, haptics). Here are some resources which introduce you to the analysis of EEG and physiological signal processing in general.

EEG glossary

To understand EEG terminology and report formats, refer to the comprehensive glossary by clinical electroencephalographers: EEG Glossary.

Setting up EEG recordings

Watch this video tutorial on setting up advanced brain monitoring electrodes for EEG recording:

Feel free to drop by the lab for more information.

EEG analysis steps

For those who are starting with EEG analysis, it can be overwhelming to go across a lot of steps. Here is almost a standard procedure for a lot of EEG analysis. Each block is a research area by itself. There are a lot of papers on each block. Do not get scared once you start the analysis of some EEG data, these steps will become second to your nature. Do read below given paper about how others are doing these steps. We use MNE-Python to implement different block functionalities. If you want more information, refer to Mike X Cohen's lectures

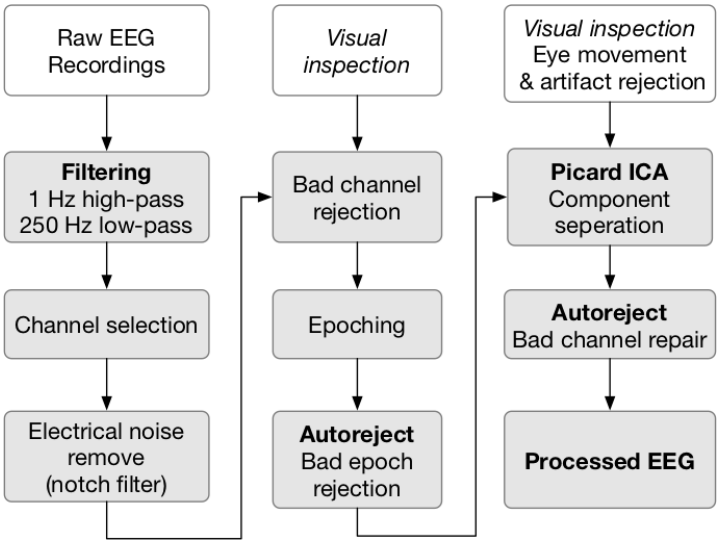


Figure 1: eeg-analysis

MNE (MNE-Python)

We extensively use MNE-Python for EEG data analysis. Resources to get you started:

- Basic MNE-Python tutorial
- Intermediate tutorials
- Join the mailing list for updates and community support.

EEG databases

Here are some databases on which you are apply your MNE-Python skills.

> **Tip**
>
> The best way to learn a (Python) package is to use it on your own data and see what are the capabilities

Pick up some data from this repository: meagmohit/EEG-Datasets. Use MNE-Python do some analysis and that is how you get introduced to analysis of EEG.

> **Tip**
>
> If you have doubts, do drop by to iHuman lab and ask any questions!

Key papers

Explore these essential papers on EEG data analysis:

- Best Practices in Data Analysis and Sharing in Neuroimaging using MEEG
- Reproducible MEG/EEG Group Study With the MNE Software
- EEG artifact removal—state-of-the-art and guidelines

GitHub links

Explore these repositories and resources for EEG and signal processing:

Analysis related repositories

- pyprep
- autoreject
- eegfaster
- neurodsp
- BioSPPy
- EMG-Signal-Processing-Library
- Python-for-Signal-Processing

Useful libraries and links

- Best Python Libraries for Psychology
- NeuroDataDesign
- EEG Notebooks
- OpenTools
- SWIPE4ICA for EEG artifact identification
- Labeling Tool

Deep learning with physiological signals

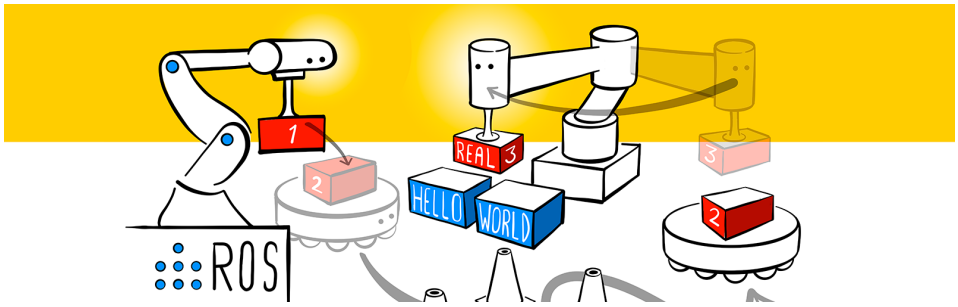
- ARL EEG Models
- DeepEEG
- Braindecode
- MNE-Torch

This guide aims to provide you with foundational resources and tools to excel in EEG research. Don't hesitate to visit the iHuman lab for further assistance and hands-on training. Happy researching!

ROS resources

Another major framework, we use in the lab is ROS. ROS, short for Robot Operating System, is an open-source framework designed to simplify the development of robotic systems. Despite its name, ROS is not an operating system itself; rather, it provides a collection of tools and libraries that help manage hardware abstraction, low-level device control, and message-passing between processes.

> **Task**
>
> Complete the free ourse on ROS: Hello (Real) World with ROS – Robot Operating System



Lab resources

Item	Units	Description	Documentation
Panda Robot	1	Advanced robotic platform for research and interaction.	Panda Robot Docs
Lamda GPU Computers	2	High-performance computers equipped with GPUs.	Lamda GPU Computers Docs
EEG Headset	1	Device for capturing brain activity through EEG signals.	EEG Headset Docs
EEG Software	1	Software for processing and analyzing EEG data.	EEG Software Docs
Eye Tracking	2	Equipment for tracking eye movements and gaze patterns.	Eye Tracking Docs
EMG Device	2	Device for measuring electrical activity in muscles.	EMG Device Docs
AR/VR Headset	1	Head-mounted display for augmented and virtual reality.	AR/VR Headset Docs
Small Drones	50	Lightweight unmanned aerial vehicles for various experiments.	Small Drones Docs
Depth Sensors	2	Sensors for measuring depth and distance in environments.	Depth Sensors Docs
Lab Computers	5	Standard desktop computers for general lab use.	Lab Computers Docs
Lab Equipment	5	Various equipment used in the laboratory for experiments.	Lab Equipment Docs
On-board Computers	3	Micro controllers for on-board processing in experiments.	On-board Computers Docs
Laptops	3	Portable laptops for mobile computing needs.	Laptops Docs

Checklist and signature page

Congrats on making towards the end of the document. Please sign the below document and send it to ihuman.research.lab@gmail.com